
Menu-Driven C Program to Prove Commutativity of Matrix Addition and Non-Commutativity of Matrix Multiplication using Variable Length 2-D Arrays

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Aim

To write a menu-driven C program, using separate functions, that:

- Accepts two matrices of variable (user-defined) size and proves that Matrix Addition is Commutative, i.e. $A + B = B + A$.
- Accepts two matrices of variable (user-defined) size and proves that Matrix Multiplication is NOT Commutative, i.e. $A \times B$ is generally not equal to $B \times A$.
- Provides a menu with an option to Exit the program.

2. Theory

2.1 Commutative Property of Matrix Addition

If A and B are two matrices of the same order (same number of rows and columns), then their sum is obtained by adding the corresponding elements of A and B. Since ordinary addition of real numbers is commutative ($a + b = b + a$ for any two numbers), it follows that adding corresponding elements in either order gives the same result. Hence, for any two matrices A and B of the same order:

$$A + B = B + A$$

This means matrix addition is always commutative, regardless of the values in the matrices.

2.2 Non-Commutative Property of Matrix Multiplication

Matrix multiplication is defined differently from addition: the element in row i, column j of the product is the sum of the products of the elements of row i of A with the corresponding elements of column j of B (the dot product of a row and a column). Because this operation depends on the specific arrangement of rows and columns, changing the order of multiplication generally changes which numbers get multiplied together and summed. Therefore, for most matrices A and B:

$$A \times B \neq B \times A$$

This program uses two square matrices of the same order $n \times n$ (so that both $A \times B$ and $B \times A$ are defined and of the same size, making them directly comparable) and shows, with a numeric example, that the two products are different.

3. Algorithm

- Step 1: Start the program and display the menu with three choices.
- Step 2: Read the user's choice.
- Step 3: If choice = 1, call `proveAdditionCommutative()`: read the order (rows, columns) of two matrices A and B of the same size, read their elements, compute $A + B$ and $B + A$ using `addMatrices()`, compare the two results using `isEqualMatrix()`, and display the conclusion.

- Step 4: If choice = 2, call `proveMultiplicationNotCommutative()`: read the order n of two square matrices A and B , read their elements, compute $A \times B$ and $B \times A$ using `multiplyMatrices()`, compare the two results, and display the conclusion.
- Step 5: If choice = 3, terminate the program.
- Step 6: Repeat Steps 2-5 until the user selects Exit.

4. Functions Used

- `createMatrix(rows, cols)` - dynamically allocates memory for a matrix of the given size (variable length 2-D array).
- `inputMatrix(mat, rows, cols, name)` - reads the elements of a matrix from the user.
- `displayMatrix(mat, rows, cols, name)` - prints a matrix in a neat, tabular form.
- `addMatrices(A, B, rows, cols)` - returns a new matrix equal to $A + B$.
- `multiplyMatrices(A, B, r1, c1, c2)` - returns a new matrix equal to $A \times B$.
- `isEqualMatrix(A, B, rows, cols)` - compares two matrices element-by-element and returns 1 if identical, 0 otherwise.
- `proveAdditionCommutative()` - drives Menu Option 1 end-to-end.
- `proveMultiplicationNotCommutative()` - drives Menu Option 2 end-to-end.
- `freeMatrix(mat, rows)` - releases dynamically allocated memory (good programming practice).

5. Source Code (matrix_commutative.c)

```

1  /*
2  * -----
3  * Program   : Menu-driven proof of Matrix Addition & Multiplication
4  *             Commutativity using variable size 2-D arrays
5  *
6  * Name      : Rudra Pratap Singh
7  * SAP ID    : 590032811
8  * Batch     : 16
9  *
10 * Description:
11 *   1. Matrix Addition      -> Prove  $A + B = B + A$  (Commutative)
12 *   2. Matrix Multiplication -> Prove  $A * B \neq B * A$  (Not Commutative)
13 *   3. Exit
14 *
15 * All operations are implemented as separate functions and the
16 * matrices are created dynamically (variable length / size, decided
17 * by the user at run time) using malloc().
18 * -----
19 */
20
21 #include <stdio.h>
22 #include <stdlib.h>
23
24 /* ----- Function Prototypes ----- */
25 int** createMatrix(int rows, int cols);
26 void freeMatrix(int **mat, int rows);
27 void inputMatrix(int **mat, int rows, int cols, char name);
28 void displayMatrix(int **mat, int rows, int cols, char name);
29
30 int** addMatrices(int **A, int **B, int rows, int cols);
31 int** multiplyMatrices(int **A, int **B, int r1, int c1, int c2);
32 int isEqualMatrix(int **A, int **B, int rows, int cols);
33
34 void proveAdditionCommutative(void);
35 void proveMultiplicationNotCommutative(void);
36
37 /* -----
38 *   main() : Displays the menu and calls the appropriate function
39 * ----- */
40 int main(void)
41 {
42     int choice;
43
44     printf("=====\n");
45     printf("  MATRIX ADDITION & MULTIPLICATION - COMMUTATIVITY\n");
46     printf("=====\n");
47

```

```

48     do
49     {
50         printf("\n----- MENU ----- \n");
51         printf("1. Prove Matrix Addition is Commutative\n");
52         printf("2. Prove Matrix Multiplication is NOT Commutative\n");
53         printf("3. Exit\n");
54         printf("----- \n");
55         printf("Enter your choice (1-3): ");
56
57         if (scanf("%d", &choice) != 1)
58         {
59             printf("Invalid input! Exiting...\n");
60             break;
61         }
62
63         switch (choice)
64         {
65             case 1:
66                 proveAdditionCommutative();
67                 break;
68
69             case 2:
70                 proveMultiplicationNotCommutative();
71                 break;
72
73             case 3:
74                 printf("\nExiting the program. Thank you!\n");
75                 break;
76
77             default:
78                 printf("\nInvalid choice! Please enter 1, 2 or 3.\n");
79         }
80
81     } while (choice != 3);
82
83     return 0;
84 }
85
86 /* -----
87  * createMatrix() : Dynamically allocates memory for a rows x cols
88  *                  matrix (variable length 2-D array)
89  * ----- */
90 int** createMatrix(int rows, int cols)
91 {
92     int **mat = (int **) malloc(rows * sizeof(int *));
93     for (int i = 0; i < rows; i++)
94         mat[i] = (int *) malloc(cols * sizeof(int));
95
96     return mat;

```

```
97 }
98
99 /* -----
100 * freeMatrix() : Releases the dynamically allocated memory
101 * ----- */
102 void freeMatrix(int **mat, int rows)
103 {
104     for (int i = 0; i < rows; i++)
105         free(mat[i]);
106     free(mat);
107 }
108
109 /* -----
110 * inputMatrix() : Reads elements of a matrix from the user
111 * ----- */
112 void inputMatrix(int **mat, int rows, int cols, char name)
113 {
114     printf("\nEnter %d elements for Matrix %c (row-wise):\n", rows * cols, name);
115     for (int i = 0; i < rows; i++)
116     {
117         for (int j = 0; j < cols; j++)
118         {
119             printf("%c[%d][%d] = ", name, i, j);
120             scanf("%d", &mat[i][j]);
121         }
122     }
123 }
124
125 /* -----
126 * displayMatrix() : Prints a matrix in a neat tabular form
127 * ----- */
128 void displayMatrix(int **mat, int rows, int cols, char name)
129 {
130     printf("\nMatrix %c (%d x %d):\n", name, rows, cols);
131     for (int i = 0; i < rows; i++)
132     {
133         printf("\t");
134         for (int j = 0; j < cols; j++)
135             printf("%6d ", mat[i][j]);
136         printf("\n");
137     }
138 }
139
140 /* -----
141 * addMatrices() : Returns a new matrix = A + B (same dimensions)
142 * ----- */
143 int** addMatrices(int **A, int **B, int rows, int cols)
144 {
145     int **sum = createMatrix(rows, cols);
146     for (int i = 0; i < rows; i++)
```

```

147         for (int j = 0; j < cols; j++)
148             sum[i][j] = A[i][j] + B[i][j];
149
150     return sum;
151 }
152
153 /* -----
154  * multiplyMatrices() : Returns a new matrix = A(r1 x c1) * B(c1 x c2)
155  * ----- */
156 int** multiplyMatrices(int **A, int **B, int r1, int c1, int c2)
157 {
158     int **prod = createMatrix(r1, c2);
159
160     for (int i = 0; i < r1; i++)
161     {
162         for (int j = 0; j < c2; j++)
163         {
164             prod[i][j] = 0;
165             for (int k = 0; k < c1; k++)
166                 prod[i][j] += A[i][k] * B[k][j];
167         }
168     }
169     return prod;
170 }
171
172 /* -----
173  * isEqualMatrix() : Compares two matrices of the same size
174  *                   Returns 1 if equal, 0 otherwise
175  * ----- */
176 int isEqualMatrix(int **A, int **B, int rows, int cols)
177 {
178     for (int i = 0; i < rows; i++)
179         for (int j = 0; j < cols; j++)
180             if (A[i][j] != B[i][j])
181                 return 0;
182
183     return 1;
184 }
185
186 /* -----
187  * proveAdditionCommutative() :
188  * Takes two variable size matrices of the SAME order (rows x cols),
189  * computes A+B and B+A and shows that both results are identical,
190  * thereby proving that Matrix Addition is COMMUTATIVE.
191  * ----- */
192 void proveAdditionCommutative(void)
193 {
194     int rows, cols;
195
196     printf("\n--- MATRIX ADDITION (A + B = B + A) ---\n");

```

```

197     printf("Note: Both matrices must have the SAME order (rows x cols)\n");
198     printf("Enter number of rows    : ");
199     scanf("%d", &rows);
200     printf("Enter number of columns : ");
201     scanf("%d", &cols);
202
203     int **A = createMatrix(rows, cols);
204     int **B = createMatrix(rows, cols);
205
206     inputMatrix(A, rows, cols, 'A');
207     inputMatrix(B, rows, cols, 'B');
208
209     displayMatrix(A, rows, cols, 'A');
210     displayMatrix(B, rows, cols, 'B');
211
212     int **AplusB = addMatrices(A, B, rows, cols);
213     int **BplusA = addMatrices(B, A, rows, cols);
214
215     printf("\n--- RESULT ---\n");
216     displayMatrix(AplusB, rows, cols, 'S'); /* A + B */
217     printf("(This is A + B)\n");
218
219     displayMatrix(BplusA, rows, cols, 'T'); /* B + A */
220     printf("(This is B + A)\n");
221
222     if (isEqualMatrix(AplusB, BplusA, rows, cols))
223         printf("\n==> A + B is EQUAL to B + A.\n"
224             "==> Hence, MATRIX ADDITION IS COMMUTATIVE. (Proved)\n");
225     else
226         printf("\n==> A + B is NOT equal to B + A.\n");
227
228     freeMatrix(A, rows);
229     freeMatrix(B, rows);
230     freeMatrix(AplusB, rows);
231     freeMatrix(BplusA, rows);
232 }
233
234 /* -----
235  * proveMultiplicationNotCommutative() :
236  * Takes two variable size SQUARE matrices of the same order n x n
237  * (so that both A*B and B*A are defined and of the same size),
238  * computes A*B and B*A and shows that, in general, the two results
239  * are NOT identical, thereby proving that Matrix Multiplication is
240  * NOT COMMUTATIVE.
241  * ----- */
242 void proveMultiplicationNotCommutative(void)
243 {
244     int n;
245

```

```

246     printf("\n--- MATRIX MULTIPLICATION (A * B != B * A) ---\n");
247     printf("Note: For A*B and B*A to both be defined and comparable,\n");
248     printf("        enter two SQUARE matrices of the same order (n x n)\n");
249     printf("Enter order 'n' of the square matrices : ");
250     scanf("%d", &n);
251
252     int **A = createMatrix(n, n);
253     int **B = createMatrix(n, n);
254
255     inputMatrix(A, n, n, 'A');
256     inputMatrix(B, n, n, 'B');
257
258     displayMatrix(A, n, n, 'A');
259     displayMatrix(B, n, n, 'B');
260
261     int **AB = multiplyMatrices(A, B, n, n, n);    /* A * B */
262     int **BA = multiplyMatrices(B, A, n, n, n);    /* B * A */
263
264     printf("\n--- RESULT ---\n");
265     displayMatrix(AB, n, n, 'P');
266     printf("(This is A x B)\n");
267
268     displayMatrix(BA, n, n, 'Q');
269     printf("(This is B x A)\n");
270
271     if (!isEqualMatrix(AB, BA, n, n))
272         printf("\n==> A x B is NOT equal to B x A.\n"
273             "    ==> Hence, MATRIX MULTIPLICATION IS NOT COMMUTATIVE. (Proved)\n");
274     else
275         printf("\n==> A x B is equal to B x A for this particular input\n"
276             "    (possible for special matrices), but in general\n"
277             "    MATRIX MULTIPLICATION IS NOT COMMUTATIVE.\n");
278
279     freeMatrix(A, n);
280     freeMatrix(B, n);
281     freeMatrix(AB, n);
282     freeMatrix(BA, n);
283 }
284

```

6. Sample Output

6.1 Option 1: Matrix Addition is Commutative

Input: Two 2 x 3 matrices A and B.

```

Enter your choice (1-3): 1

--- MATRIX ADDITION (A + B = B + A) ---
Enter number of rows      : 2
Enter number of columns   : 3

Enter 6 elements for Matrix A (row-wise):
A[0][0]=1 A[0][1]=2 A[0][2]=3
A[1][0]=4 A[1][1]=5 A[1][2]=6

Enter 6 elements for Matrix B (row-wise):
B[0][0]=6 B[0][1]=5 B[0][2]=4
B[1][0]=3 B[1][1]=2 B[1][2]=1

Matrix A (2 x 3):
    1    2    3
    4    5    6

Matrix B (2 x 3):
    6    5    4
    3    2    1

--- RESULT ---

Matrix S (2 x 3):      (This is A + B)
    7    7    7
    7    7    7

Matrix T (2 x 3):      (This is B + A)
    7    7    7
    7    7    7

==> A + B is EQUAL to B + A.
==> Hence, MATRIX ADDITION IS COMMUTATIVE. (Proved)

```

6.2 Option 2: Matrix Multiplication is NOT Commutative

Input: Two 2 x 2 (square) matrices A and B.

```

Enter your choice (1-3): 2

--- MATRIX MULTIPLICATION (A * B != B * A) ---
Enter order 'n' of the square matrices : 2

```

```
Enter 4 elements for Matrix A (row-wise):
A[0][0]=1  A[0][1]=2
A[1][0]=3  A[1][1]=4

Enter 4 elements for Matrix B (row-wise):
B[0][0]=5  B[0][1]=6
B[1][0]=7  B[1][1]=8

Matrix A (2 x 2):
    1    2
    3    4

Matrix B (2 x 2):
    5    6
    7    8

--- RESULT ---

Matrix P (2 x 2):      (This is A x B)
    19    22
    43    50

Matrix Q (2 x 2):      (This is B x A)
    23    34
    31    46

==> A x B is NOT equal to B x A.
==> Hence, MATRIX MULTIPLICATION IS NOT COMMUTATIVE. (Proved)
```

6.3 Option 3: Exit

```
Enter your choice (1-3): 3

Exiting the program. Thank you!
```

7. Conclusion

The menu-driven C program was implemented successfully using separate, well-defined functions for input, display, addition, multiplication, comparison and memory management. Matrices of variable, user-defined dimensions were created dynamically at run time using `malloc()`. The sample runs confirm the two fundamental properties of matrix algebra:

- Matrix Addition is commutative: $A + B$ always equals $B + A$ for matrices of the same order.
- Matrix Multiplication is, in general, NOT commutative: $A \times B$ is not equal to $B \times A$, even for square matrices of the same order.